

裂纹板材加工问题中 四种启发式算法的详细构造

摘要

本文在同一确定性数学模型下,依次构造价值优先算法VF、价值密度优先算法VDF、动态边际收益贪心算法MG 以及边际收益引导局部搜索算法MG-LS。四种算法共享相同的候选表示、设备容量约束、木块状态编码和确定性并列规则,仅逐层升级候选评分与搜索范围:

单块价值 $\rightarrow$ 单位时间价值 $\rightarrow$ 动态边际收益 $\rightarrow$ 局部反悔与区域重构.

文中给出每种算法的候选生成、评分公式、可行性检查、逐轮更新、停止条件、伪代码和复杂度,并明确木块的身份、原始位置与邻接关系在全部算法中始终固定。

目录

1	四种算法共用的模型基础	3
1.1	集合、参数与固定位置约束	3
1.2	加工状态与方案表示	4
1.3	目标函数与等价优化部分	4
1.4	共用的单块候选集合	5
1.5	统一的确定性并列规则	5
2	VF: 单块价值优先算法	6
2.1	设计动机	6
2.2	静态候选与评分	6
2.3	详细构造过程	6
2.4	算法性质与局限	7
3	VDF: 单位加工时间价值优先算法	7
3.1	从 VF 到 VDF 的唯一核心改动	7

3.2	详细构造过程	8
3.3	改进效果与剩余局限	8
4	MG: 动态边际收益贪心算法	9
4.1	从静态密度到动态边际密度	9
4.2	单块边际收益的展开	9
4.3	动态评分与选择规则	10
4.4	详细构造过程	10
4.5	为什么必须每轮重新计算	11
4.6	剩余局限	11
5	MG-LS: 边际收益引导局部搜索算法	11
5.1	总体结构	11
5.2	第一阶段: MG 初始解	12
5.3	第二阶段: 三类局部搜索邻域	12
5.3.1	邻域总集合	12
5.3.2	SingleAdjust: 单块调整	12
5.3.3	PrecisionExchange: 精加工机会替换	12
5.3.4	EdgeRebuild: 相邻区域重构	13
5.4	邻域评价与每轮操作依据	13
5.5	完整停止条件	14
6	四种算法的逐层递进关系	14
7	复杂度与候选规模	14
7.1	VF 与 VDF	14
7.2	MG	15
7.3	MG-LS	15
8	模型口径变化时的调整说明	16
9	适合 PPT 展示的四句总结	16

1 四种算法共用的模型基础

1.1 集合、参数与固定位置约束

设木块集合为

$$\mathcal{U} = \{1, 2, \dots, n\},$$

普通加工设备集合和精密加工设备集合分别为

$$\mathcal{K}^O, \quad \mathcal{K}^P,$$

且

$$\mathcal{K}^O \cap \mathcal{K}^P = \emptyset, \quad \mathcal{K} = \mathcal{K}^O \cup \mathcal{K}^P.$$

木块之间的原始相邻关系由无向边集合

$$\mathcal{E} = \{\{u, v\} \mid u, v \in \mathcal{U}, u < v, u \text{ 与 } v \text{ 在原板材中相邻}\}$$

给出。木块 u 的相邻木块集合记为

$$\mathcal{N}_u = \{v \in \mathcal{U} \mid \{u, v\} \in \mathcal{E}\}.$$

位置不变性：木块的身份、原始空间位置和邻接关系均为固定数据。算法只决定木块是否加工、采用何种加工方式以及分配给哪台设备，绝不交换或改变木块的最终位置。即使木块被改派到另一台设备，加工结束后仍回到它自己的原始位置。

木块 u 的面积、固有价值 and 裂纹严重程度分别记为

$$A, \quad v_u, \quad CS_u.$$

设备 k 的加工速度、单位面积加工增值和时间上限分别记为

$$s_k, \quad r_k, \quad T.$$

木块 u 在设备 k 上的确定性加工时间为

$$t_{u,k} = \frac{A}{s_k} (1 + \alpha CS_u).$$

对于相邻木块对 $\{u, v\} \in \mathcal{E}$ ，若两块均采用精密加工，可获得组合增值 b_{uv} ；若恰有一块采用精密加工，则产生精度不一致损失 p_{uv} 。

1.2 加工状态与方案表示

木块 u 的加工状态为

$$q_u \in \{0, O, P\},$$

其中 0 、 O 、 P 分别表示不加工、普通加工和精密加工。设备分配变量为

$$x_{u,k} = \begin{cases} 1, & \text{木块}u \text{ 分配给设备}k, \\ 0, & \text{否则.} \end{cases}$$

普通加工和精密加工状态分别为

$$y_u^O = \sum_{k \in \mathcal{K}^O} x_{u,k}, \quad y_u^P = \sum_{k \in \mathcal{K}^P} x_{u,k},$$

并满足

$$\sum_{k \in \mathcal{K}} x_{u,k} \leq 1, \quad y_u^0 + y_u^O + y_u^P = 1.$$

一个部分或完整加工方案记为

$$S = \{(u, q, k) \mid q \in \{O, P\}, x_{u,k} = 1\}.$$

若木块 u 没有出现在 S 中, 则 $q_u = 0$, 但木块本身仍保留在原始位置。

设备 k 在方案 S 下的当前负载为

$$L_k(S) = \sum_{(u,q,k) \in S} t_{u,k}.$$

设备容量约束为

$$L_k(S) \leq T, \quad \forall k \in \mathcal{K}.$$

1.3 目标函数与等价优化部分

对于相邻木块对 $\{u, v\} \in \mathcal{E}$, 定义

$$h_{uv}(S) = y_u^P(S) y_v^P(S),$$

$$m_{uv}(S) = y_u^P(S) + y_v^P(S) - 2h_{uv}(S).$$

于是确定性目标函数为

$$\begin{aligned} Z(S) = & \sum_{u \in \mathcal{U}} v_u + \sum_{(u,q,k) \in S} Ar_k \\ & + \sum_{\{u,v\} \in \mathcal{E}} [b_{uv} h_{uv}(S) - p_{uv} m_{uv}(S) - L_{uv}^{\text{cr}}]. \end{aligned}$$

由于木块及其位置固定，

$$\sum_{u \in \mathcal{U}} v_u \quad \text{与} \quad \sum_{\{u,v\} \in \mathcal{E}} L_{uv}^{\text{cr}}$$

都是与加工决策无关的常数。算法比较两个方案优劣时，可以使用等价的决策相关目标

$$F(S) = \sum_{(u,q,k) \in S} Ar_k + \sum_{\{u,v\} \in \mathcal{E}} [b_{uv}h_{uv}(S) - p_{uv}m_{uv}(S)].$$

最终汇报板材总价值时，再将固定项加回即可。

1.4 共用的单块候选集合

单块加工候选统一写成

$$a = (u, q, k),$$

其中 u 是尚未加工的木块， $q \in \{O, P\}$ ，设备 k 必须与加工方式兼容：

$$k \in \begin{cases} \mathcal{K}^O, & q = O, \\ \mathcal{K}^P, & q = P. \end{cases}$$

给定当前方案 S ，可行单块候选集合为

$$\mathcal{A}_1(S) = \left\{ (u, q, k) \left| \begin{array}{l} q_u(S) = 0, \\ k \in \mathcal{K}^q, \\ L_k(S) + t_{u,k} \leq T \end{array} \right. \right\}.$$

若执行候选 $a = (u, q, k)$ ，新方案记为

$$S \oplus a = S \cup \{(u, q, k)\}.$$

1.5 统一的确定性并列规则

不同候选可能具有相同评分。为保证算法可复现，四种算法均使用固定的并列规则。一般按照以下顺序比较：

1. 主评分更高者优先；
2. 若主评分相同，实际增益或单块价值更高者优先；
3. 若仍相同，加工时间更短者优先；
4. 若仍相同，按木块编号、设备编号和加工方式的固定字典序选择。

因此，同一组输入参数每次都会得到相同结果。

2 VF：单块价值优先算法

2.1 设计动机

VF 是四种算法的基础基线。它只回答一个最直接的问题：

“哪个单块加工方案的表面价值最高？”

算法不考虑单位加工时间效率，也不根据当前相邻木块的加工状态动态改变评分。

2.2 静态候选与评分

先在空方案下生成全部单块候选：

$$\mathcal{A}_1^{\text{all}} = \{(u, q, k) \mid u \in \mathcal{U}, q \in \{O, P\}, k \in \mathcal{K}^q, t_{u,k} \leq T\}.$$

候选 $a = (u, q, k)$ 的单块价值为

$$g^{\text{VF}}(a) = v_u + Ar_k.$$

其中 v_u 用于体现木块本身的重要程度， Ar_k 表示设备加工带来的增值。该评分只在算法开始前计算一次。

2.3 详细构造过程

1. 初始化空方案 $S \leftarrow \emptyset$ ，并令 $L_k(S) = 0, \forall k \in \mathcal{K}$ 。
2. 枚举全部 $a \in \mathcal{A}_1^{\text{all}}$ ，计算 $g^{\text{VF}}(a)$ 。
3. 按 g^{VF} 从高到低排序，得到静态序列

$$a_{(1)}, a_{(2)}, \dots, a_{(M)}.$$

4. 从头到尾扫描该序列。对当前候选 $a_{(j)} = (u, q, k)$ ：
 - 若木块 u 已经被更早的候选安排，则跳过；
 - 若 $L_k(S) + t_{u,k} > T$ ，则跳过；
 - 否则接受该候选，令 $S \leftarrow S \oplus a_{(j)}$ ，并更新设备负载。
5. 扫描完全部候选后输出方案 S_{VF} 。

算法 1: VF 的伪代码

1. $S \leftarrow \emptyset, L_k \leftarrow 0$;
2. 构造 $\mathcal{A}_1^{\text{all}}$;
3. 对每个 $a = (u, q, k)$ 计算 $g(a) \leftarrow v_u + Ar_k$;
4. 按评分降序及统一并列规则对候选排序;
5. **for** 每个候选 $a = (u, q, k)$ **do**
 - 若 $q_u(S) \neq 0$, **continue**;
 - 若 $L_k + t_{u,k} > T$, **continue**;
 - $S \leftarrow S \oplus a$;
 - $L_k \leftarrow L_k + t_{u,k}$;
6. **return** $S_{\text{VF}} \leftarrow S$.

2.4 算法性质与局限

VF 始终保持以下可行性不变量:

$$\sum_{k \in K} x_{u,k} \leq 1, \quad L_k(S) \leq T.$$

其主要局限是: 高价值候选可能占用很长的加工时间, 从而挤占多个“价值略低但耗时很短”的候选; 同时, 固定评分无法体现相邻木块的组合增值和精度不一致损失。

3 VDF: 单位加工时间价值优先算法**3.1 从 VF 到 VDF 的唯一核心改动**

VDF 完全保留 VF 的候选表示、静态排序、可行性检查和扫描更新过程, 只将评分从“单块价值”改为“单位加工时间价值”:

$$\underbrace{v_u + Ar_k}_{\text{VF}} \implies \underbrace{\frac{v_u + Ar_k}{t_{u,k}}}_{\text{VDF}}.$$

因此

$$g^{\text{VDF}}(a) = \frac{v_u + Ar_k}{t_{u,k}}.$$

3.2 详细构造过程

1. 初始化 $S \leftarrow \emptyset$ 和全部设备负载。
2. 生成与 VF 完全相同的候选集合 $\mathcal{A}_1^{\text{all}}$ 。
3. 对每个候选 $a = (u, q, k)$ 计算

$$g^{\text{VDF}}(a) = \frac{v_u + Ar_k}{t_{u,k}}.$$

4. 按价值密度从高到低一次性排序。
5. 依次扫描候选，并使用与 VF 相同的“木块未安排且设备容量充足”判定接受候选。
6. 扫描结束后输出 S_{VDF} 。

算法 2: VDF 的伪代码

1. $S \leftarrow \emptyset, L_k \leftarrow 0$;
2. 构造 $\mathcal{A}_1^{\text{all}}$;
3. 对每个 $a = (u, q, k)$ 计算

$$g(a) \leftarrow \frac{v_u + Ar_k}{t_{u,k}};$$
4. 按评分降序及统一并列规则对候选排序;
5. **for** 每个候选 $a = (u, q, k)$ **do**
 - 若 $q_u(S) \neq 0$, **continue**;
 - 若 $L_k + t_{u,k} > T$, **continue**;
 - $S \leftarrow S \oplus a$;
 - $L_k \leftarrow L_k + t_{u,k}$;
6. **return** $S_{\text{VDF}} \leftarrow S$.

3.3 改进效果与剩余局限

VDF 能够避免“价值高但耗时过长”的候选过早占满设备，更符合有限工期下的资源利用逻辑。但是，它仍然使用预先计算的静态评分。当一个相邻木块被选为精密加工后，其他相邻木块的真实收益会发生变化，VDF 却不会重新计算排序。

4 MG: 动态边际收益贪心算法

4.1 从静态密度到动态边际密度

MG 保留 VDF 的“收益除以加工时间”结构，但把静态的单块价值改成候选加入当前方案后带来的当前收益评分：

$$\underbrace{\frac{v_u + Ar_k}{t_{u,k}}}_{\text{VDF}} \implies \underbrace{g^{\text{MG}}(a | S)}_{\text{MG}}.$$

候选 a 相对于当前方案 S 的当前收益来自新旧方案目标值之差：

$$\text{gain}(a | S) = F(S \oplus a) - F(S).$$

由于木块固有价值 and 跨块裂纹损失为常数，它们在差分中自动抵消。

4.2 单块边际收益的展开

对候选 $a = (u, q, k)$ ，只有木块 u 的加工增值以及所有与 u 相连的邻接边可能发生变化。因此

$$\text{gain}(a | S) = Ar_k + \sum_{v \in \mathcal{N}_u} [b_{uv}d_{uv}^h - p_{uv}d_{uv}^m].$$

其中

$$d_{uv}^h = h_{uv}(S \oplus a) - h_{uv}(S),$$

$$d_{uv}^m = m_{uv}(S \oplus a) - m_{uv}(S).$$

若 $q = O$ ，木块 u 的精密状态仍为0，所以邻接状态不变：

$$\text{gain}((u, O, k) | S) = Ar_k.$$

若 $q = P$ ，可进一步写成

$$\text{gain}((u, P, k) | S) = Ar_k + \sum_{\substack{v \in \mathcal{N}_u \\ y_v^P(S)=1}} (b_{uv} + p_{uv}) - \sum_{\substack{v \in \mathcal{N}_u \\ y_v^P(S)=0}} p_{uv}.$$

其含义是：

- 若相邻木块 v 已经精密加工，则新增 b_{uv} ，同时消除原先的精度不一致损失 p_{uv} ；
- 若相邻木块 v 尚未精密加工，则新产生精度不一致损失 p_{uv} 。

4.3 动态评分与选择规则

MG 的评分为

$$g^{\text{MG}}(a \mid S) = \frac{\text{gain}(a \mid S)}{t_{u,k}}.$$

每一轮重新生成当前可行候选集合，并选择

$$a^* = \arg \max_{a \in \mathcal{A}_1(S)} g^{\text{MG}}(a \mid S).$$

只有当

$$\text{gain}(a^* \mid S) > \varepsilon$$

时才执行该候选，其中 $\varepsilon > 0$ 是数值容差。

4.4 详细构造过程

1. 初始化 $S \leftarrow \emptyset$ 。
2. 根据当前木块状态和设备剩余容量生成 $\mathcal{A}_1(S)$ 。
3. 对每个 $a \in \mathcal{A}_1(S)$ 计算当前收益和评分 $g(a)$ 。
4. 选择评分最大的候选 a^* 。
5. 若不存在候选，或 $\text{gain}(a^* \mid S) \leq \varepsilon$ ，停止构造。
6. 否则执行 a^* ，更新方案和设备负载，然后返回步骤 2。

算法 3: MG 的伪代码

1. $S \leftarrow \emptyset$;
2. **while true do**
 - 2.1. 生成当前可行候选集合 $\mathcal{A}_1(S)$;
 - 2.2. 若 $\mathcal{A}_1(S) = \emptyset$, **break**;
 - 2.3. 对每个 $a = (u, q, k) \in \mathcal{A}_1(S)$:
$$\text{gain}(a) \leftarrow F(S \oplus a) - F(S), \quad g(a) \leftarrow \frac{\text{gain}(a)}{t_{u,k}};$$
 - 2.4. $a^* \leftarrow \arg \max_{a \in \mathcal{A}_1(S)} g(a)$;
 - 2.5. 若 $\text{gain}(a^*) \leq \varepsilon$, **break**;
 - 2.6. $S \leftarrow S \oplus a^*$;
3. **return** $S_{\text{MG}} \leftarrow S$.

4.5 为什么必须每轮重新计算

执行一个精密加工候选后，至少会改变以下信息：

- 相关设备的剩余容量；
- 相邻边上的 h_{uv} 和 m_{uv} ；
- 相邻木块下一轮的精密加工边际收益；
- 当前可行候选集合。

因此，MG 不能像 VF 和 VDF 一样只排序一次。

4.6 剩余局限

MG 每轮只加入一个木块。可能存在两个相邻木块单独加入时均不划算，但同时进行精密加工时能够获得较大的组合增值，即

$$\text{gain}(u \mid S) \leq 0, \quad \text{gain}(v \mid S) \leq 0,$$

却有

$$\text{gain}(\{u, v\} \mid S) > 0.$$

单块贪心无法直接发现这种“只有联合选择才有价值”的机会，也不能撤销早期已经接受的决定。

5 MG-LS：边际收益引导局部搜索算法

5.1 总体结构

MG-LS: Marginal-Gain-Guided Local Search.

该算法不再设计额外的相邻对联合构造阶段，而是直接以 MG 的结果作为初始解，再通过局部搜索修正早期贪心决策：

$$\text{MG-LS} = \text{MG 初始解} + \text{针对性局部搜索}.$$

对应流程为

$$\emptyset \xrightarrow{\text{MG 动态构造}} S_{\text{MG}} \xrightarrow{\text{局部反悔与重构}} S_{\text{MG-LS}}.$$

MG 的优势是每轮都能感知当前方案下的真实边际收益；但它只有“加入”操作，不能撤销已经占用设备容量的早期选择。因此其核心缺陷是：

MG 能发现当前最优候选，但早期贪心决策一旦接受就无法反悔。

MG-LS 的作用正是允许在保持木块位置和邻接关系不变的前提下，对加工方式、设备分配和局部精加工区域进行有限调整。

5.2 第一阶段：MG 初始解

首先运行前一节的动态边际收益贪心算法：

$$S^{(0)} = S_{\text{MG}}.$$

由于后续局部搜索只接受正增益调整，因此必然有

$$F(S_{\text{MG-LS}}) \geq F(S_{\text{MG}}).$$

5.3 第二阶段：三类局部搜索邻域

5.3.1 邻域总集合

给定当前可行方案 S ，MG-LS 的局部搜索邻域定义为

$$\mathcal{N}_{\text{MG-LS}}(S) = \mathcal{N}_{\text{SA}}(S) \cup \mathcal{N}_{\text{EX}}(S) \cup \mathcal{N}_{\text{ER}}(S).$$

三类邻域分别对应单块调整、精加工机会替换和相邻木块重加工。

5.3.2 SingleAdjust：单块调整

选择一个木块 u ，重新枚举其加工状态和兼容设备：

$$(u, q_u, k_u) \longrightarrow (u, q'_u, k'_u), \quad q'_u \in \{0, O, P\}.$$

该邻域覆盖取消加工、普通加工改精密加工、精密加工改普通加工、未加工木块新增加工，以及同类型设备之间的重新分配。

5.3.3 PrecisionExchange：精加工机会替换

选择当前精加工木块 u 和当前非精加工木块 v ：

$$q_u = P, \quad q_v \neq P.$$

联合执行

$$u : P \rightarrow O/0, \quad v : O/0 \rightarrow P.$$

也就是说，该邻域释放一个被早期选择占用的精加工机会，再交给当前更适合精加工的木块。

5.3.4 EdgeRebuild: 相邻木块重加工

对每条相邻边 $\{u, v\} \in \mathcal{E}$ ，先释放不超过 ρ 个当前已经精加工的木块，腾出精密加工资源；再尝试让该相邻边两端同时精加工：

$$R \subseteq P(S), \quad |R| \leq \rho.$$

局部候选方案可以概括为

$$S' = (S \setminus S_{R \cup \{u, v\}}) \cup \hat{S}_{R \cup \{u, v\}},$$

其中要求 $q'_u = q'_v = P$ ，而 R 中木块可降为普通加工或不加工。该邻域可概括为“释放旧精加工资源，换取相邻木块联合精加工机会”，用于形成更合理的连续精加工局部区域。通常取 $\rho = 1$ 或 $\rho = 2$ ，避免候选数量过大。

5.4 邻域评价与每轮操作依据

对所有邻域生成的每个具体候选方案 S' ，依次执行：

1. 检查每个木块是否至多分配给一台设备；
2. 检查设备类型与加工方式是否兼容；
3. 检查全部设备容量约束；
4. 重新计算受影响的加工增值和邻接价值；
5. 计算调整方案增益

$$\text{gain}_{\text{LS}}(S' \mid S) = F(S') - F(S).$$

采用全邻域最优改进策略：

$$S^* = \arg \max_{S' \in \mathcal{N}_{\text{MG-LS}}(S), S' \text{ 可行}} \text{gain}_{\text{LS}}(S' \mid S).$$

因此，每轮执行什么操作完全由当前方案下所有具体可行候选的增益决定，而不是预先规定某一轮必须执行哪一种邻域操作。

若

$$\text{gain}_{\text{LS}}(S^* \mid S) > \varepsilon,$$

则更新 $S \leftarrow S^*$ ；否则算法到达当前邻域下的局部最优并停止。最大局部搜索轮数 R_{LS} 只是计算时间保护上限。

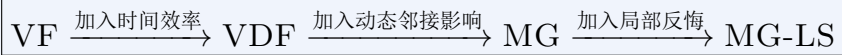
算法 4: MG-LS 边际收益引导局部搜索

1. 运行 MG, 得到初始方案 $S \leftarrow S_{\text{MG}}$;
2. **for** $r = 1, 2, \dots, R_{\text{LS}}$ **do**
 - 2.1. 生成 SingleAdjust、PrecisionExchange 和 EdgeRebuild 的全部具体候选;
 - 2.2. 删除所有违反唯一分配、设备兼容或容量约束的候选;
 - 2.3. 对每个剩余候选 S' 计算 $\text{gain}_{\text{LS}}(S') = F(S') - F(S)$;
 - 2.4. 若不存在候选, **break**;
 - 2.5. $S^* \leftarrow \arg \max_{S'} \text{gain}_{\text{LS}}(S')$;
 - 2.6. 若 $\text{gain}_{\text{LS}}(S^*) \leq \varepsilon$, **break**;
 - 2.7. $S \leftarrow S^*$;
3. **return** $S_{\text{MG-LS}} \leftarrow S$.

5.5 完整停止条件

MG-LS 的构造阶段直接沿用 MG 的停止条件; 局部搜索阶段在以下情况下停止:

- 全部具体邻域候选均不可行;
- 最佳可行邻域候选的目标增量不大于 ε ;
- 达到最大局部搜索轮数 R_{LS} 。

6 四种算法的逐层递进关系**7 复杂度与候选规模**

记

$$n = |\mathcal{U}|, \quad K = |\mathcal{K}|, \quad K_P = |\mathcal{K}^P|, \quad e = |\mathcal{E}|.$$

7.1 VF 与 VDF

静态单块候选数量至多为

$$M_1 = O(nK).$$

算法	主评分	保留上一层的内容	本层新增能力
VF	$v_u + Ar_k$	统一候选、容量检查与贪心接受框架	建立最简单的单块价值基线
VDF	$\frac{v_u + Ar_k}{t_{u,k}}$	VF 的静态候选、排序和更新过程	用加工时间归一化，衡量资源效率
MG	$g^{\text{MG}}(a \mid S)$	VDF 的“收益/时间”结构与单块候选	每轮重算真实边际收益，感知相邻状态
MG-LS	$F(S') - F(S)$	MG 的动态构造结果	单块调整、精加工机会替换和相邻木块重加工

表 1: 四种算法的递进关系

候选生成需要 $O(nK)$ ，排序需要

$$O(nK \log(nK)),$$

顺序扫描需要 $O(nK)$ 。因此两种算法的主复杂度均为

$$O(nK \log(nK)).$$

7.2 MG

每轮至多枚举 $O(nK)$ 个单块候选，并计算候选关联边上的增量。构造阶段最多接受 n 个木块，因此可保守写为

$$O(n(nK + e)).$$

若预先建立每个木块的关联边表，则单个候选只需访问该木块的邻接边，实际运行通常明显快于完整重算目标函数。

7.3 MG-LS

MG-LS 的第一阶段直接调用 MG，因此构造阶段复杂度沿用 MG。局部搜索阶段中各邻域的候选数量上界可概括为：

邻域	候选数量级
SingleAdjust	$O(nK)$
PrecisionExchange	$O(n^2K)$
EdgeRebuild	$O\left(\sum_{j=0}^{\rho} \binom{n}{j} eK_P^2\right)$

当默认 $\rho = 1$ 时,

$$\text{EdgeRebuild} = O(neK_P^2).$$

因此, MG-LS 的主要计算开销来自 PrecisionExchange 和 EdgeRebuild 邻域。实际实现中可通过木块规模阈值、候选预筛选、增量式目标更新和最大迭代次数限制运行时间。

8 模型口径变化时的调整说明

本文与当前确定性模型一致, 允许

$$q_u \in \{0, O, P\},$$

即木块可以不加工。若后续题目明确改为“每个木块都必须加工”, 则应将

$$\sum_{k \in \mathcal{K}} x_{u,k} \leq 1$$

改为

$$\sum_{k \in \mathcal{K}} x_{u,k} = 1, \quad \forall u \in \mathcal{U}.$$

此时需要同步进行三项修改:

1. 构造算法不能在部分方案上最终停止, 必须加入修复或回溯过程, 直至全部木块完成分配;
2. 局部搜索删除纯 Drop 和纯 Add 邻域;
3. Swap 改为两个已加工木块之间的加工方式或设备联合重分配, 始终保持每个木块处于已加工状态。

无论采用哪一种口径, 木块的原始位置和邻接关系都始终不允许改变。

9 适合 PPT 展示的四句总结

1. **VF**: 优先安排单块价值最高的加工方案, 建立基础价值基线。
2. **VDF**: 在 VF 基础上除以加工时间, 优先安排单位时间价值更高的方案。
3. **MG**: 将静态价值替换为当前方案下的真实边际收益, 每轮重算相邻组合影响。
4. **MG-LS**: 以 MG 结果为初始解, 通过单块调整、精加工机会替换和相邻木块重加工修正早期贪心决策。